



UNITED INTERNATIONAL UNIVERSITY (UIU)
DIGITAL SIGNAL PROCESSING LABORATORY
COURSE CODE EEE 3310

EXPERIMENT NO. 1

**INTRODUCTION TO CONTINUOUS TIME SIGNAL TO DISCRETE TIME SIGNAL
CONVERSION**

Prerequisite

Before starting this section, student should know:

1. A fundamental knowledge on MATLAB
2. A theoretical knowledge on sampling, reconstruction and quantization.

Lab Outcome

After Finishing this lab student should be able to:

1. Understand signal sampling and reconstruction and their implementation using MATLAB.
2. Understand quantization and its implementation using MATLAB.

1.1 Sampling of Continuous Time (CT) Signal

Introduction:

Signals available in our environment are mostly analog and continuous time. As a result, we cannot use them for digital signal processing as DSP deals with discrete time signals. As DSP is unable to handle continuous time signals, we need to take samples of the signal to make it processable with DSP. Again, as the amplitude of the signal of any fractional value. Digital computers are unable to sense all infinitesimal fractional numbers. As a result, we need to quantize the discrete samples to process them with DSP.

Sampling:

Sampling is a process of converting a continuous time signal into a discrete time signal. Digital devices including computers need to process signals and data in discrete time. To convert a continuous signal into a discrete signal, samples of the signal are taken periodically which is known as uniform sampling.

If $y(t)$ is a continuous time signal, the corresponding discrete signal will be $y(nT)$ where $n=1,2,3,4,....$ and T is the sampling interval. We can also write the discrete signal as $y(n)$ with sampling period of T .

Sampling Frequency:

Sampling frequency is the number of samples taken per minute by the continuous to discrete converter. If the sampling period is T , the sampling frequency will be, $f_s = \frac{1}{T}$.

If the sampling frequency is increased the number of samples taken per minute will be increased. Again, if the sampling frequency is decreased the number of samples taken per minute will be decreased.

The Nyquist-Shanon Sampling Theorem:

The Nyquist–Shannon sampling theorem is a fundamental result in the field of information theory, in particular telecommunications and signal processing. The theorem is commonly called Shannon's sampling theorem.

The theorem states that exact reconstruction of a continuous-time baseband signal from its samples is possible if the signal is bandlimited and the sampling frequency is greater than twice the signal bandwidth. The theorem also leads to an effective reconstruction formula.

Example 1.1:

Consider an analog sinusoidal signal $x_1(t) = 2 \cos 2\pi(10)t$. The signal is sampled at a rate $F_s = 40$ Hz. Now, sample this signal using the sampling frequency F_s and plot the signal and sampled signal for 5 cycles.

Sampling signals using MATLAB code:

```
%This is Lab 1.1: Sampling
clear all; close all;
%%Plotting of a sinusoidal signal
F=10; %Frequency of 10 Hz
A=2; %Amplitude is 2 volts
theta=0; %Phase shift=0;
T=1/F; %time period
delT= 0.001; %Sample Spacing
duration= 5*T; %For 5 cycles
N= round(duration/delT); %Total number of samples per cycle
t= (0:N-1)*delT; %Time Array
x_1= A*cos((2*pi*F*t)+theta);

figure
plot(t,x_1) grid on
%%Sampling the sine wave
Fs= 8*F; Ts= 1/Fs;
Ns= 40; % 5 cycles with 8 times of frequency
t_sampled= (0:Ns-1)*Ts;
x_1_sampled= A*cos((2*pi*F*t_sampled)+theta);
hold on
stem(t_sampled, x_1_sampled);
title('5 cycles of continuous time and sampled Cosine signal')
ylabel('Amplitude of Sinusoidal'); xlabel('Time(seconds)');
legend({'x_1(t)', 'x_1(n)'})
```

Output of Example 1(a):

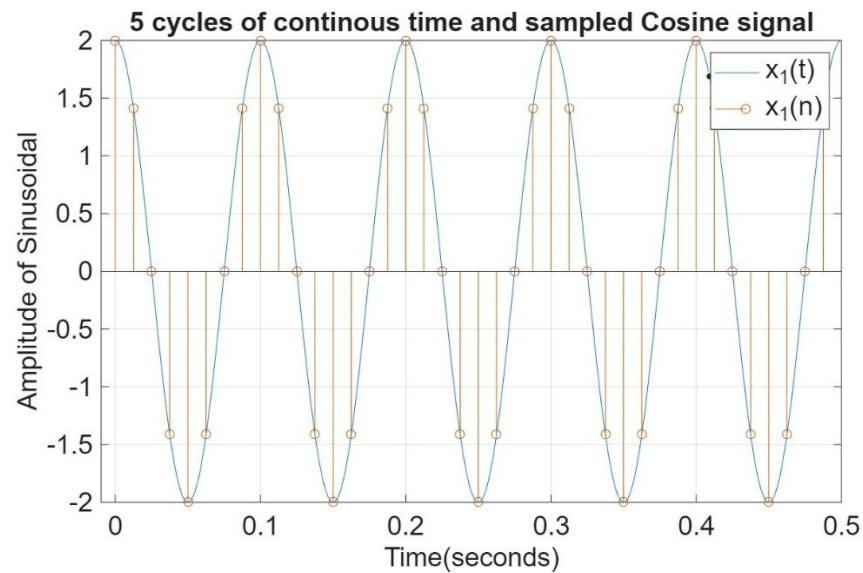


Figure 1: Sampling of an analog signal of 10Hz

1.2 Observing Aliasing Effect

Example 1.2:

Consider two analog sinusoidal signals $x_1(t) = \cos 2\pi(10)t$ and $x_2(t) = \cos 2\pi(50)t$. Both signals are sampled at a rate $F_s = 40$ Hz for 5 cycles. Compare the plot with the plot in Example 1.1

Observing Aliased signals using MATLAB code:

```
%This is Lab 1.2: Aliasing
clear all; close all;
F1=10;F2=50;
Fs=40;
delT= 0.001; %Sample Spacing
n=0:500; %Number of points
x_1=cos(2*pi*F1*n*delT);
x_2=cos(2*pi*F2*n*delT);

plot(n*delT,x_1, 'r');
hold on;
plot(n*delT,x_2,'g');

Ts=1/Fs;
N=round(1/Ts); %Number of samples for 40 Hz sampling rate
n=0:N/2; %Sample point array
x1=cos(2*pi*F1*n*Ts); %Sampled signal at 40 Hz
x2=cos(2*pi*F2*n*Ts); %Sampled signal at 40 Hz

hold on
plot(n*Ts,x1, 'o');
hold on
plot(n*Ts,x2, 'o');
```

```

title(['Aliasing effect on two sinusoidal with 10Hz and 50 Hz Frequency'])
ylabel('Amplitude of Sinusoidal')
xlabel('Time(seconds)')
legend({'x_1(t)', 'x_2(t)', 'Sample for Fs=40'})

```

Output of Example 1.2:

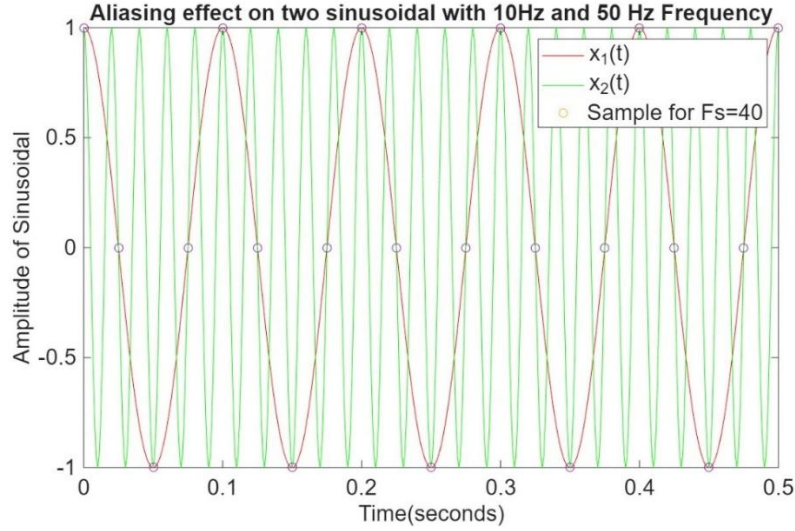


Figure 2: Observing Aliasing Effect on two harmonic Sinusoidal signals

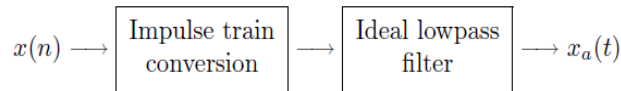
1.3 Reconstruction of Sampled Discrete Time (DT) Signal using Sinc Interpolation

From the sampling theorem and the preceding examples, it is clear that if we sample band-limited $x_a(t)$ above its Nyquist rate, then we can reconstruct $x_a(t)$ from its samples $x(n)$. This reconstruction can be thought of as a 2-step process:

- (i) First the samples are converted into a weighted impulse train.

$$\sum_{n=-\infty}^{\infty} x(n)\delta(t - nT_s) = \dots + x(-1)\delta(n + T_s) + x(0)\delta(t) + x(1)\delta(n - T_s) + \dots$$

- (ii) Then the impulse train is filtered through an ideal analog lowpass filter band-limited to the $[-F_s/2, F_s/2]$ band.



This two-step procedure can be described mathematically using an interpolating formula is an interpolating function. We observe that this ideal interpolation is not practically feasible because the entire system is noncausal and hence not realizable. $x_a(t) = \sum_{n=-\infty}^{\infty} x(n)\text{sinc}[F_s(t - nT_s)]$, where $\text{sinc}(x) = \frac{\sin \pi x}{\pi x}$

Example 1.3:

Consider two analog sinusoidal signals $x_1(t) = \cos 2\pi(10)t$ and $x_2(t) = \cos 2\pi(50)t$. Both signals are sampled at a rate $F_s = 40$ Hz.

Now, (a) Reconstruct the sampled signal using x_1 , x_2 and $x_3 = x_1 + x_2$ using Sinc interpolation and analyze the results and

(b) Reconstruct the sampled signal using x_1 using Sinc interpolation at $T_s = 0.01$, $T_s = 0.05$ and $T_s = 0.1$ and analyze the results.

Reconstruction of signals from using MATLAB code:

```
%%This is Lab 1.3(a): Reconstruction
```

```
clear all; close all;
```

```
A=1; %Amplitude is 1 volts
```

```
t=0:0.001:1; %Time array for 1 second
```

```
%%Sampling the sine wave
```

```
Fs= 40; %Sampling Frequency
```

```
Ts=1/Fs;
```

```
N1=round(1/Ts); %Number of samples per cycle
```

```
n1=0:N1;
```

```
x1=A*cos(2*10*pi*n1*Ts); %sampled x1 with 10 Hz freq
```

```
x2=A*cos(2*50*pi*n1*Ts); %sampled x2 with 50 Hz freq
```

```
x3= x1+x2; %sampled x1+x2
```

```
%%Reconstruction:
```

```
x_1=x1*sinc(Fs*(ones(length(n1),1)*t-(n1*Ts)'*ones(1,length(t))));
```

```
x_2=x2*sinc(Fs*(ones(length(n1),1)*t-(n1*Ts)'*ones(1,length(t))));
```

```
x_3=x3*sinc(Fs*(ones(length(n1),1)*t-(n1*Ts)'*ones(1,length(t))));
```

```
figure
```

```
subplot(3, 1, 1); plot(t,x_1); title('Reconstructed signal x1 with 10 Hz frequency at  
40 Hz sampling Frequency')
```

```
ylabel('Amplitude'); xlabel('Time(seconds)'); legend('x_1');
```

```
subplot(3, 1, 2); plot(t,x_2); title('Reconstructed signal x2[Aliased] with 50 Hz  
frequency at 40 Hz sampling Frequency')
```

```
ylabel('Amplitude'); xlabel('Time(seconds)'); legend('x_2');
```

```
subplot(3, 1, 3); plot(t,x_3); title('Reconstructed signal x3=x1+x2[Aliased] at 40 Hz  
sampling Frequency')
```

```
ylabel('Amplitude'); xlabel('Time(seconds)'); legend('x_3');
```

```
fontsize(14,"points")
```

Output of Example 1.3(a):

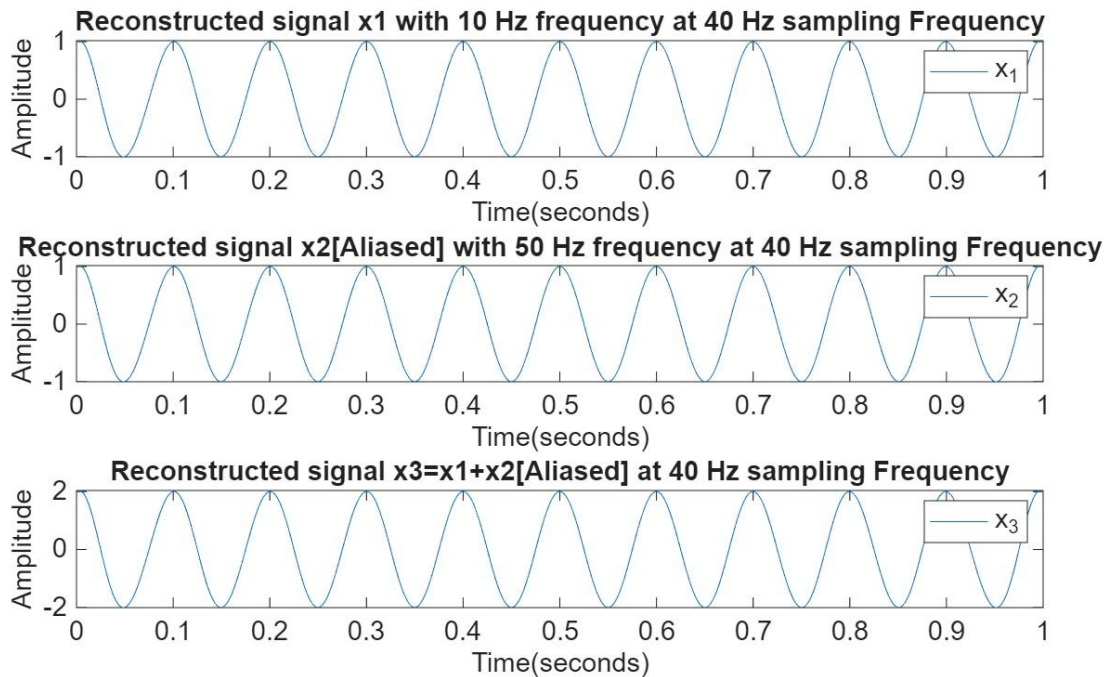


Figure 3: Reconstructed analog signal using Sinc Interpolation

Reconstruction of signals from using MATLAB code:

```
%%This is Lab 1.3(b): Reconstruction
clear all; close all
%% Sampling
t=0:0.001:1;
xa=cos(20*pi*t); % f=10
Ts1=0.01; N1=round(1/Ts1); n1=0:N1;
x1=cos(20*pi*n1*Ts1);
Ts2=0.05; N2=round(1/Ts2); n2=0:N2;
x2=cos(20*pi*n2*Ts2);
Ts3=0.1; N3=round(1/Ts3); n3=0:N3;
x3=cos(20*pi*n3*Ts3);

figure
subplot(3,1,1);plot(t,xa,n1*Ts1,x1,'o');
axis([0,1,-1.1,1.1]); ylabel('x1(n)');
title('Sampling of x_{a}(t)using Ts=0.01');
subplot(3,1,2); plot(t,xa,n2*Ts2,x2,'o');
axis([0,1,-1.1,1.1]); ylabel('x2(n)');
title('Sampling of x_{a}(t)using Ts=0.05');
subplot(3,1,3); plot(t,xa,n3*Ts3,x3,'o');
axis([0,1,-1.1,1.1]); ylabel('x3(n)');
title('Sampling of x_{a}(t)using Ts=0.1');

%% Reconstruction:
Ts=0.01; Fs=1/Ts;
```

```

xa1=x1*sinc(Fs*(ones(length(n1),1)*t-(n1*Ts)'*ones(1,length(t))));
Ts=0.05; Fs=1/Ts;
xa2=x2*sinc(Fs*(ones(length(n2),1)*t-(n2*Ts)'*ones(1,length(t))));
Ts=0.1; Fs=1/Ts;
xa3=x3*sinc(Fs*(ones(length(n3),1)*t-(n3*Ts)'*ones(1,length(t))));

figure
subplot(3,1,1); plot(t,xa1);
axis([0,1,-1.1,1.1]); ylabel('x_{a}(t)'); xlabel('Time(s)');
title('Reconstruction of x_{a}(t) when Ts=0.01 ');
subplot(3,1,2); plot(t,xa2);
axis([0,1,-1.1,1.1]); ylabel('x_{a}(t)'); xlabel('Time(s)');
title('Reconstruction of x_{a}(t) when Ts=0.05 ');
subplot(3,1,3);plot(t,xa3);
axis([0,1,-1.1,1.1]); ylabel('x_{a}(t)'); xlabel('Time(s)');
title('Reconstruction of x_{a}(t) when Ts=0.1 '); fontsize(14, "points")

```

Output of Example 1.3(b):

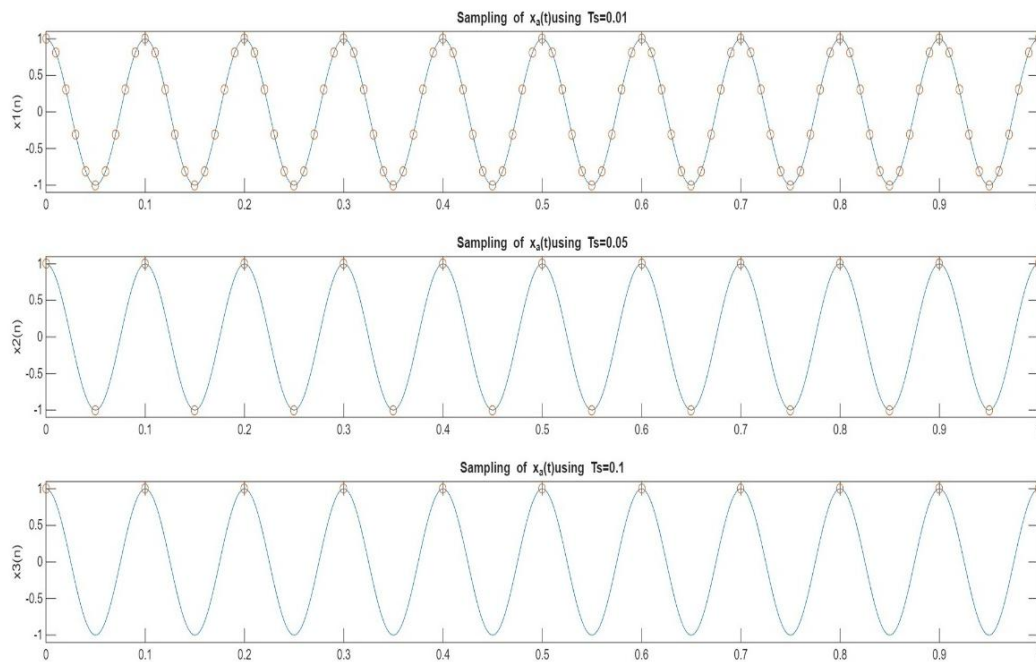


Figure 4: Sampled analog signal using different sampling frequency

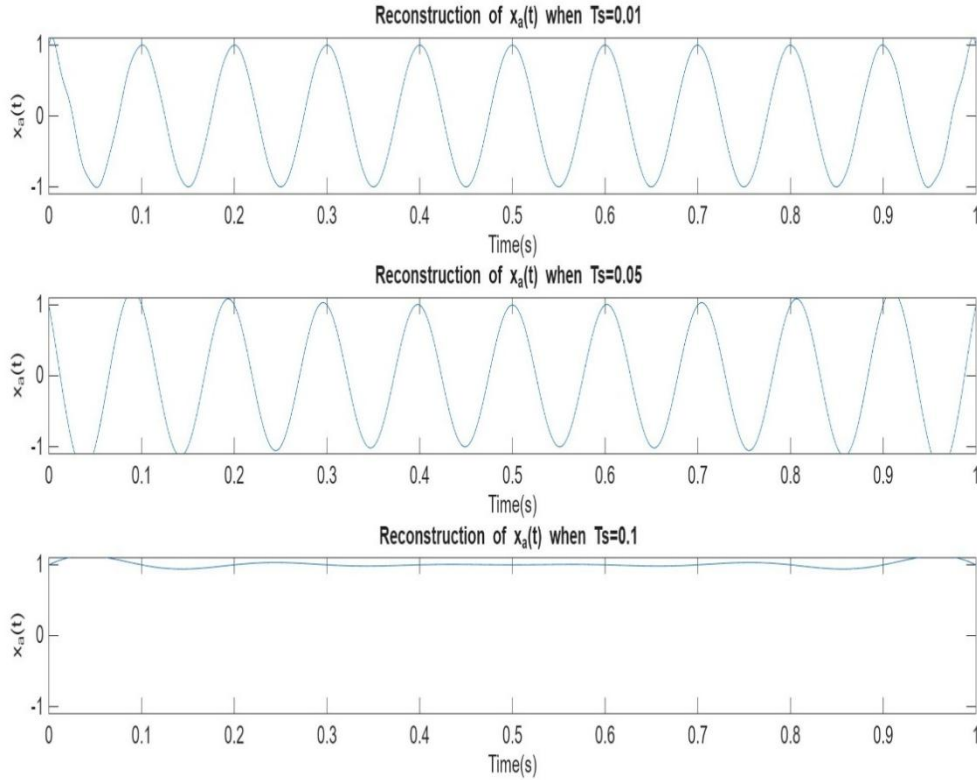


Figure 5: Reconstructed signal using Sinc Interpolation for different sampling frequency

1.4 Quantization of DT signal and Reconstruction of Quantized Signal

We denote the quantizer operation on the samples $x(n)$ as $Q[x(n)]$ and let $x_q(n)$ denote the sequence of quantized samples at the output of the quantizer. The quantized sample $x_q(n)$ is determined as, $x_q(n) = \Delta \times L_{signal}$.

If X_{min} and X_{max} represent the minimum and maximum values of $x(n)$ and L is the number of quantization levels, then $\Delta = \frac{x_{max}-x_{min}}{L-1}$. We define the dynamic range of the signal as $(X_{min} - X_{max})$, quantization step as Δ , and quantization level as L . Here, $L_{signal} = \text{round}\left(\frac{x(n)-X_{min}}{\Delta}\right)$

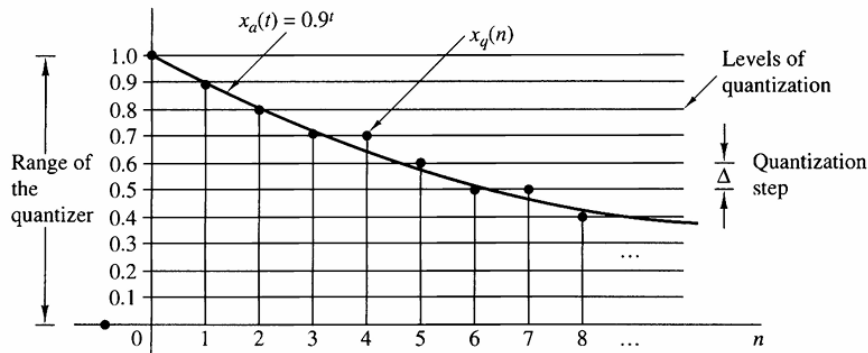
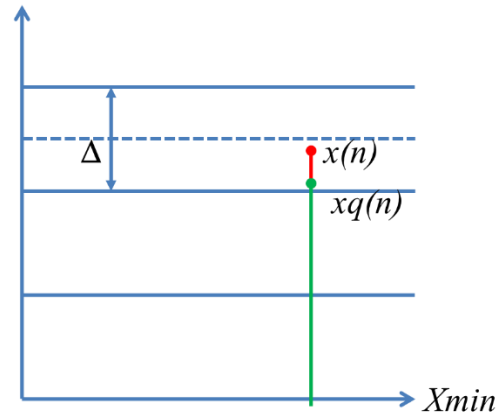


Figure 6: Illustration of Quantization level, analog signal and samples



$$x_q(n) = \left[\text{round}\left(\frac{x(n) - X_{min}}{\Delta}\right) \right] \Delta + X_{min}$$

Figure 7: Illustration of Amplitude Level Determination of $x_q(n)$

Example 1.4:

Consider an analog sinusoidal signal $x_1(t) = \cos 2\pi(10)t$. Sampling rate, $F_s = 40$ Hz. Quantize the signals x_1 for the following bits: $b = 2$ and $b = 4$.

Quantization of signals using MATLAB code:

Step 1: Save the function as “uquant”

```
function y=uquant(x,L)
%x= original sequence
% L=Number of Levels

maxval=max(max(x));
minval=min(min(x));

stepsize=(maxval-minval)/(L-1);

y=round((x-minval)/stepsize)*stepsize+minval;
```

Step 2: Call the function as “uquant” to quantize signal

```
%%This is Lab 1.4: Quantization
clear all; close all;
%% Sampling
t=0:0.001:1;
A= 1;
Fs= 40;
Ts=1/Fs;
N1=round(1/Ts); n1=0:N1;
t_sample= n1*Ts;
x1=A*cos(20*pi*n1*Ts);

%% Quantization
b1= 2; b2=4; %number of bits
L1= 2^b1; L2= 2^b2; %number of Levels
```

```

x1q_L1= uquant(x1,L1); %quantized signal x1 with bit b=2
x1q_L2= uquant(x1,L2); %quantized signal x1 with bit b=4

figure
stem(t_sample,x1,'x');
hold on
stem(t_sample,x1q_L1,'r');
title('Quantization of x_1 signal with b=2')
legend('Sampled x_1', 'Quantized x_1');
ylabel('Amplitude'); xlabel('Time(s)'); fontsize(16, "points")
figure
stem(t_sample,x1,'x');
hold on
stem(t_sample,x1q_L2,'r');
title('Quantization of x_1 signal with b=4')
legend('Sampled x_1', 'Quantized x_1');
ylabel('Amplitude'); xlabel('Time(s)'); fontsize(16, "points")

```

Output of Example 1.4:

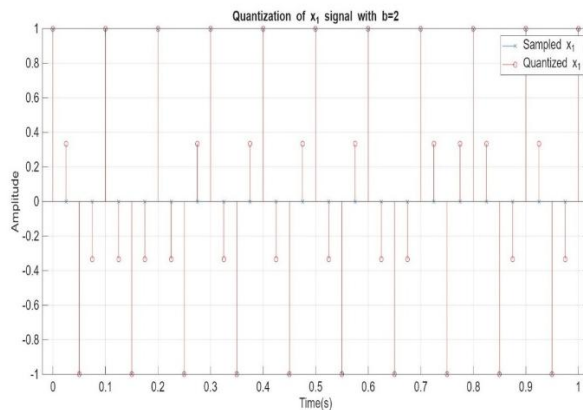


Figure 8: Samples of sampled signal and Quantized signal for bit 2

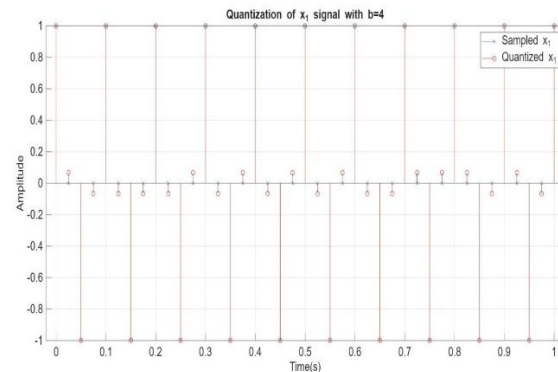


Figure 9: Samples of sampled signal and quantized signal for bit 4

1.5 Calculation of Signal-To-Quantization Noise Ratio (SQNR)

For a analog sinusoidal signal $x(t) = A \cos 2\pi f t$, sampled at sampling frequency F_s satisfying the sampling theorem, quantization is the only error in the Analog to Digital conversion process. The reconstructed signal after quantization using sinc interpolation is $x_q(n)$. Quantization error, $e_q(n) = x(n) - x_q(n)$.

From the textbook (Digital Signal Processing, 4th Edition, John G. Proakis & Dimitris G. Manolakis) equation 1.4.32 is following:

$$SQNR(dB) = 10 \log_{10}(SQNR) = 1.76 + 6.02b$$

This implies that the SQNR increase approximately 6 dB for every bit added to the word length, that is for each doubling of the quantization levels.

To calculate the SQNR from measured data, one can find the instantaneous error as $e_q(n) = x(n) - x_q(n)$. Then one can find the average squared error as the Mean Square Error (MSE)= $\text{mean}(e_q^2(n))$.

Example 1.5:

Consider two analog sinusoidal signals (i) $x_1(t) = \cos 2\pi(10)t$. The signal is sampled at a rate $F_s = 40$ Hz. Now, calculate the Quantization error and SQNR for quantization at $b=2$.

Quantization of signals using MATLAB code:

```
%This is Lab 1.5: Calculating Quantization Noise(error) and SQNR
clear all; close all;
%% Sampling
A= 1;Fs= 40; Ts=1/Fs; N1=round(1/Ts); n1=0:N1; x1=A*cos(20*pi*n1*Ts);
%% Quantization
b= 2; %number of bits L= 2^b; %number of Levels
xq= uquant(x1,L); %quantized signal x1 with bit b=2
%%Calculation of SQNR(dB)
quant_error= x1-xq; %Noise(error) calculation
figure
stem(n1*Ts, x1); hold on
stem(n1*Ts, xq, 'g'); hold on
plot(n1*Ts, quant_error, 'r'); ylabel('Amplitude'); xlabel('Time(s)')
legend('x_1', 'x_q', 'e_q'); fontsize(18, "points")
sq_error= (x1-xq).^2; %Noise Power MSE= mean(sq_error); %Average of Noise Power
SQNR= 10*log10((A^2/2)/MSE) %Signal-to-Quantization Noise Ratio(SQNR)
```

Output of Example 1.5:

SQNR_Calculation

SQNR =

9.6497

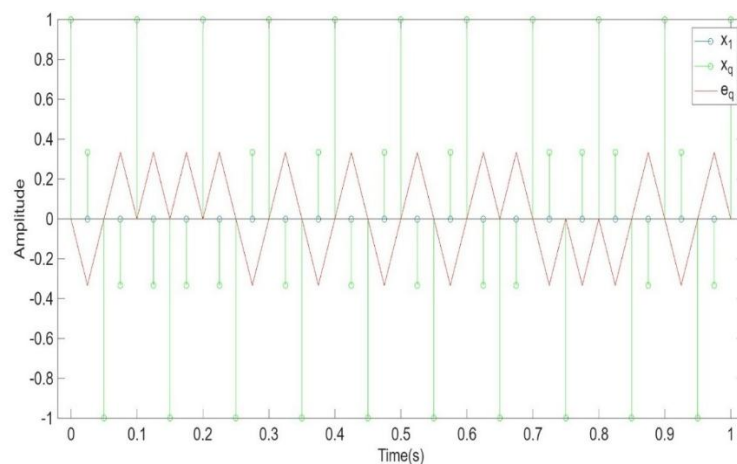


Figure 10: Sampled Signal, Quantized signal and quantization error

1.6 Complete System Implementation: Analog to Digital Conversion (ADC) and Digital to Analog Conversion (DAC)

Example 1.5:

Consider an analog signal $x_a(t) = \cos(20\pi t)$, $0 \leq t \leq 1$. It is sampled at $T_s = 0.01\text{sec}$ intervals to obtain $x(n)$. Now,

- Sample the signal and plot $x(n)$.
- Quantize the sampled signals with $b=8$ to convert analog signal into digital bit value.
- Reconstruct the quantized signals using sinc interpolation to convert digital signal to analog.
- Calculate the Quantization error and SQNR for quantization at $b=8$.
- Analyze all the outputs

Complete System Implementation ADC to DAC using

```
%%Lab 1.6: Complete ADC and DAC with bit value 2
clear all; close all
%% ADC: Sampling
t=0:0.001:1; A=2;
xa=A*cos(20*pi*t); % f=10

Ts1=0.01;
N1=round(1/Ts1); n1=0:N1;
x1=A*cos(20*pi*n1*Ts1); % sampled signal

figure
plot(t,xa,n1*Ts1,x1,'o');axis([0,1,-2.2,2.2]);
ylabel('x1(n)'); title('Sampling of x_{a}(t) using Ts=0.01');
fontsize(16,"points")

%% ADC: Quantization
b= 8; %number of bits
L= 2^b; %number of levels
xq= uquant(x1,L); %quantized signal x1

figure
stem(x1,'x')
hold on
stem(xq,'r'); legend('x_1', 'x_q')
title('Quantized signal x_{q}(n)'); fontsize(16,"points")

%% DAC: Reconstruction of Quantized Signal
Ts=0.01; Fs=1/Ts;
xa1=xq*sinc(Fs*(ones(length(n1),1)*t-(n1*Ts))*ones(1,length(t))));

figure
plot(t,xa1); axis([0,1,-2.5,2.5]);
ylabel('x_{q}(t)'); title('Reconstruction of x_{q}(t) when Ts=0.01 ');
fontsize(16,"points")

%%DAC: Calculating SQNR for Quality Assurance
quant_error= x1-xq; %Noise (error) Calculation
```

```
sq_error= (x1-xq).^2; %Power of Noise
MSE= mean(sq_error); %Average Power of Noise
SQNR= 10*log10((A^2/2)/MSE) %SQNR in dB
```

```
figure
plot(quant_error,'r')
title('Quantized Error of  $x_{\{1\}}(n)$ ');fontsize(16,"points")
```

Output of Example 1.6:

SQNR = 51.8905

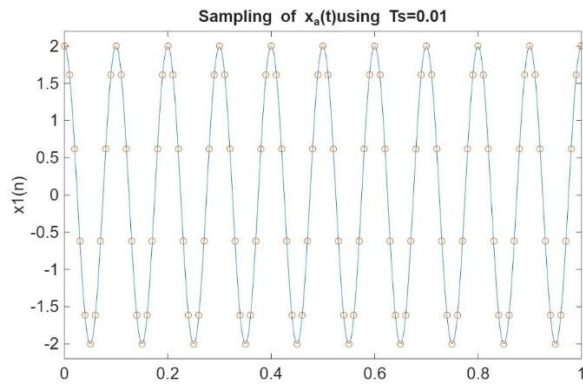


Figure 11: Analog signal and samples after sampling

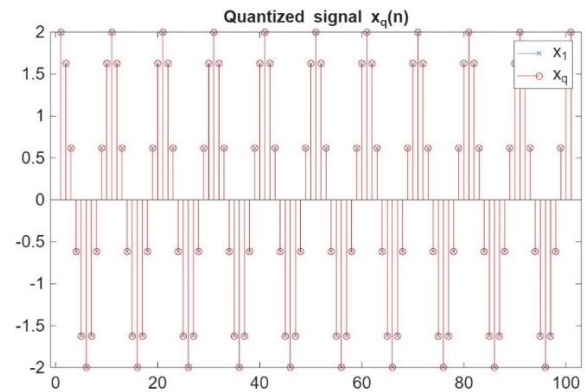


Figure 12: Quantized Signal (ADC)

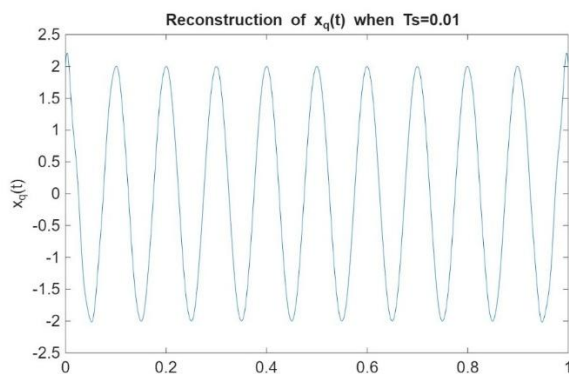


Figure 13: Reconstructed analog signal after quantization (DAC)

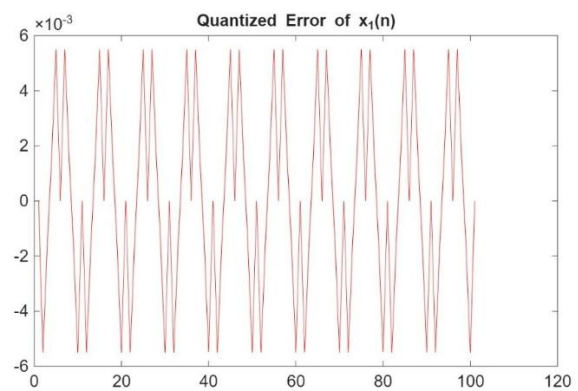


Figure 14: Quantization Error of the system

1.7 Exercise

HW1.01:

Practice plotting problem: Consider two analog sinusoidal signals $x_1(t) = 10 \cos 2\pi(10)t$ and $x_2(t) = \cos 2\pi(50)t$. Let, $x_3 = x_1 + x_2$. Now, plot x_1 , x_2 and x_3 for $0 \leq t < 1$

HW1.02:

Given a sinusoid waveform with a frequency of 100 Hz,

$$x(t) = 4.5 \sin(2\pi \times 100t)$$

The signal $x(t)$ is sampled at $F_s = 8000\text{Hz}$ and quantized with bits, $b=6$. Write a MATLAB program to quantize $x(t)$ and plot the quantized signal.

Answer the following questions:

- How many numbers of samples are there in 1 cycle of the sinusoidal?
- What is the value of sample spacing or time interval?
- What is the Nyquist Rate for the given sinusoidal?
- How many levels are there for Quantization?
- What is the SQNR?

HW1.03:

Given a sinusoid waveform with a frequency of 100 Hz,

$$x(t) = 4.5 \sin(2\pi \times 2500t)$$

The signal $x(t)$ is sampled at $F_s = 8000\text{Hz}$ and quantized with bits 2, 4 and 8.

- Write a MATLAB program to quantize $x(t)$ for each bit and plot the quantized signal. Calculate the SQNR using Mean Square Error (MSE)
- Write a MATLAB program calculate the SQNR for bits 2, 4 and 8 using the following equation:

$$SQNR(dB) = 1.76 + 6.02b$$

- Compare your results from (a) and (b) for a specific bit value and write your comments.
- Let the required SQNR in an ADC be 60dB. What bit value would you choose for the ADC?

Hint: You may use “for loop” in HW1.03 as a trial and error approach.